

0.76.5. Un déclencheur en (0, 0) : le C file à gauche

Le 0.76.4, plus un C : il apparaît au bord droit, sur la ligne 3, et file vers la gauche ; au bord, il repart de droite.

1. Ce qu'il faut comprendre

C'est le 0.76.4, plus un C (`ALPHABET[2]`).

Ce qui est nouveau ici : un mouvement vers la GAUCHE, qui recommence. À chaque tic du chronomètre, le C efface sa case, recule d'une colonne (`cx--`), et se repose. Arrivé à la colonne 0, il **repart de la colonne 19** :

```
if (cx == 0) { cx = 19; } else { cx--; }.
```

if ... else avec accolades : si `cx` vaut 0, on fait le premier bloc (repartir à droite) ; sinon, le second (reculer d'une case). Jamais les deux.

Les fonctions de cette leçon — et le tuto de chacune

- `#include <deplace_croix>` → [0.66.3. La fonction deplace_croix\(\) — suivre la croix, case par case](#)
- `#include <ALPHABET>` → [0.1.4. ALPHABET — les lettres de la police](#)
- `#include <nombre>` → [0.8.3. La fonction nombre\(\) — écrire un nombre](#)
- `#include <poser>` → [0.1.3. La fonction poser\(\) — poser une tuile](#)
- `#include <un_pas>` → [0.29.4. La fonction un_pas\(\) — un seul pas, sans attendre](#)
- `#include <effacer>` → [0.16.3. La fonction effacer\(\) — effacer des cases](#)

2. Le programme, morceau par morceau

Le programme entier de la leçon, découpé comme l'œil le lit. Il fait **1809 octets** de cartouche.

Ce que le compilateur y a gravé, nommé — chaque morceau, sa taille, qui l'a demandé — et ce qu'il a laissé, parce que rien ne l'appelle :

Gravé dans la cartouche :

```
démarrage      109 o ← la console, à l'allumage - la mise en route de la console
deplace_croix() 217 o ← main() - une fonction fournie par le langage
un_pas()        117 o ← main() - une fonction fournie par le langage
main()         474 o ← démarrage - ta fonction
VBlank         8 o ← la console, à chaque image - compte les images, 60 fois par seconde
AttendreImage   28 o ← image() - attend l'image suivante
AttendreVBlank  16 o ← démarrage - attend que l'écran soit libre
AttendreAcces   12 o ← EffacerCases, EcrireNombre - attend de pouvoir écrire une case
AdresseCase     10 o ← effacer(), poser() - trouve l'adresse d'une case de l'écran
EffacerCases    12 o ← effacer() - efface des cases
EcrireNombre    33 o ← nombre() - écrit un nombre en chiffres
CopierTuiles    18 o ← démarrage - copie les lettres et les dessins en mémoire vidéo
EffacerCarte    14 o ← démarrage - vide les deux cartes de l'écran
LireManette     37 o ← bouton() - lit les huit boutons
Tuiles         704 o ← CopierTuiles - la police entière
Lettres gravées : toutes (poser() reçoit un numéro calculé)
```

Pas gravé (jamais appelé) : AvancerAirs, AttendreFinVBlank, EcrireTexte, Diviser, Reste, Multiplier, DecalerGauche, DecalerDroite, RangerLutins, InstallerTransfert, Hasard, EffacerFond, EffacerPanneau, DonneesTransfert, Notes

Fonctions du langage appelées :

poser()	× 6	57 o sur place, dans deplace_croix(), un_pas(), main()	+ AdresseCase (10 o)
effacer()	× 3	45 o sur place, dans deplace_croix(), un_pas(), main()	+ AdresseCase (10 o), EffacerCases (12 o)
bouton()	× 4	20 o sur place, dans deplace_croix()	+ LireManette (37 o)
nombre()	× 2	20 o sur place, dans main()	+ EcrireNombre (33 o)
image()	× 1	3 o sur place, dans main()	+ AttendreImage (28 o)
images()	× 1	2 o sur place, dans deplace_croix()	

Tes fonctions appelées :

un_pas()	× 8	176 o sur place, dans main()
deplace_croix()	× 1	19 o sur place, dans main()

1. CE QUE LE PROGRAMME RACONTE DE LUI-MÊME

```
#include <deplace_croix> // une tuile qui suit la croix, case par case
#include <ALPHABET>      // les lettres de la police : ALPHABET[0] est le A
#include <nombre>         // écrit un nombre en chiffres
#include <poser>          // pose une tuile sur une case du fond
#include <un_pas>         // fait faire un seul pas à une tuile
#include <effacer>        // efface des cases, ou tout le fond
```

2. UNE VARIABLE — X

```
uint8_t x = 10; // le A part de (10, 0)
```

3. UNE VARIABLE — Y

```
uint8_t y = 0;
```

4. UNE VARIABLE — ACTIF

```
uint8_t actif = 0; // 0 : pas encore ; 1 : le A est passé par (0, 0)
```

5. UNE VARIABLE — BX

```
uint8_t bx = 10; // le B : sa colonne...
```

6. UNE VARIABLE — BY

```
uint8_t by = 8; // ...et sa ligne. Il apparaît au milieu, (10, 8).
```

7. UNE VARIABLE — BPAS

```
uint8_t bpas = 0; // où il en est dans son tour : du pas 0 au pas 15
```

8. UNE VARIABLE — BIMAGES

```
uint8_t bimages = 0; // le chronomètre du B, en images
```

9. UNE VARIABLE — BX2

```
uint8_t bx2 = 14; // le second B : il part du coin opposé, (14, 12)...
```

10. UNE VARIABLE — BY2

```
uint8_t by2 = 12;
```

11. UNE VARIABLE — BPAS2

```
uint8_t bpas2 = 8;    // ...donc déjà à mi-tour : il commence par aller à gauche
```

12. UNE VARIABLE — CX

```
uint8_t cx = 19;      // le C : il apparaît en (19, 3), au bord droit
```

13. UNE VARIABLE — CY

```
uint8_t cy = 3;
```

14. LE POINT D'ENTRÉE : C'EST ICI QUE LA CONSOLE ARRIVE

```
int main() {
    while (true) {
        image();
        deplace_croix(x, y, ALPHABET[0], 250); // le A suit la croix
        nombre(0, 17, x);                      // sa position, en direct
        nombre(4, 17, y);

        // Le déclencheur : le A arrive en (0, 0), et actif vaut encore 0.
        if (x == 0 && y == 0 && actif == 0) {
            actif = 1;                          // une seule fois : ensuite actif vaut 1
            poser(10, 8, ALPHABET[1]);          // le B apparaît au milieu
            poser(14, 12, ALPHABET[1]);        // le second B, au coin opposé
            poser(19, 3, ALPHABET[2]);         // le C apparaît au bord droit
        }

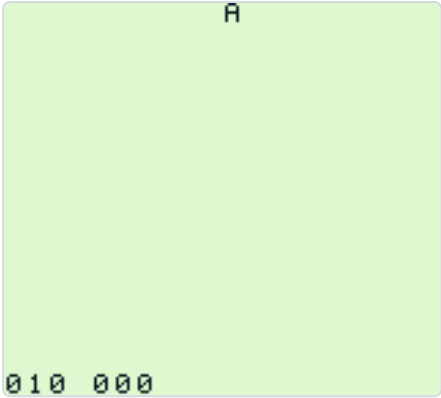
        // Les mouvements, une fois le déclencheur passé.
        if (actif == 1) {
            bimages++;                          // le chronomètre avance
            if (bimages == 5) {                 // toutes les 5 images : un pas
                bimages = 0;
                if (bpas < 4) { un_pas(bx, by, ALPHABET[1], 1, 0); } // pas 0 à 3 : à droite
                else if (bpas < 8) { un_pas(bx, by, ALPHABET[1], 0, 1); } // pas 4 à 7 : en bas
                else if (bpas < 12) { un_pas(bx, by, ALPHABET[1], -1, 0); } // pas 8 à 11 : à gauche
                else { un_pas(bx, by, ALPHABET[1], 0, -1); } // pas 12 à 15 : en haut
                bpas++;
                if (bpas == 16) { bpas = 0; } // le tour est fini : on recommence
                if (bpas2 < 4) { un_pas(bx2, by2, ALPHABET[1], 1, 0); } // pas 0 à 3 : à droite
                else if (bpas2 < 8) { un_pas(bx2, by2, ALPHABET[1], 0, 1); } // pas 4 à 7 : en bas
                else if (bpas2 < 12) { un_pas(bx2, by2, ALPHABET[1], -1, 0); } // pas 8 à 11 : à gauche
                else { un_pas(bx2, by2, ALPHABET[1], 0, -1); } // pas 12 à 15 : en haut
                bpas2++;
                if (bpas2 == 16) { bpas2 = 0; } // le tour est fini : on recommence
                effacer(cx, cy, 1); // le C : un pas à GAUCHE...
                if (cx == 0) { cx = 19; } else { cx--; } // ...et au bord gauche, il repart de droite
                poser(cx, cy, ALPHABET[2]);
            }
        }
    }
}
```

3. Ce qui se passe, étape par étape

La cartouche a été compilée et exécutée pour de bon ; voici son écran aux instants qui comptent. Une image identique à la précédente n'est pas montrée — un programme qui ne bouge pas n'a qu'une seule image, et c'est la vérité sur ce programme.



la toute première image — la console vient de démarrer — image 1



un tiers de seconde plus tard : le premier dessin est posé — image 20

4. Ce qu'on doit voir

En (0, 0), les deux B tournent, et un C file vers la gauche en haut de l'écran, sans fin.

5. Ce que la console répond

Ces contrôles ne sont pas décoratifs : ce sont ceux que `verifier-tuto.mjs` exige de cette leçon. Ils viennent d'être joués, et voici ce qu'ils ont mesuré.

- ✓ le C est apparu
- ✓ il va vers la GAUCHE (11 → 3)
- ✓ à sa place, ligne 3